EduPlatform AI

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Aman Verma	**EN22CS301110**
Anjali Jaiswal	**EN22CS301132**
Ankit Parmar	**EN22CS301135**
Ansh Laad	**EN22CS301141**
Apoorva Gupta	**EN22CS301190**
Arya Bhawsar	**EN22CS301209**

Under the Guidance of

Prof.(Dr.) Hemlata Patel

Prof. Ajeet Singh Rajput

Prof. Akshay Saxena

MEDICAPS UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

EduPlatform AI

A Project-II Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Aman Verma	**EN22CS301110**
Anjali Jaiswal	**EN22CS301132**
Ankit Parmar	**EN22CS301135**
Ansh Laad	**EN22CS301141**
Apoorva Gupta	**EN22CS301190**
Arya Bhawsar	**EN22CS301209**

Under the Guidance of

Prof.(Dr.) Hemlata Patel

Prof. Ajeet Singh Rajput

Prof. Akshay Saxena

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

JAN-JUNE 2026

Report Approval

The project work **"EduPlatform AI"** is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approved any statement made, opinion expressed, or conclusion drawn there in but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled **"EduPlatform AI"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and engineering completed under the supervision of **Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput** Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Aman Verma EN22CS301110

Anjali Jaiswal EN22CS301132

Ankit Parmar EN22CS301135

Ansh Laad EN22CS301141

Apoorva Gupta EN22CS301190

Arya Bhawsar EN22CS301209

Certificate

We, **Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput** certify that the project entitled **"EduPlatform AI"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Aman Verma, Anjali Jaiswal, Ankit Parmar, Ansh Laad, Apoorva Gupta, Arya Bhawsar** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

Prof.(Dr.) Hemlata Patel
Computer Science and Engineering

Prof. Ajeet Singh Rajput
Computer Science and Engineering

Prof. Akshay Saxena
Computer Application

Medicaps University, Indore

_____

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express my deepest gratitude to Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya,** Associate Dean, Faculty of Engineering, Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

We express our heartfelt gratitude to our Internal Guide, Prof. Akshay Saxena, Prof. Hemlata Patel, Prof. Ajeet Singh Rajput without whose continuous help and support, this project would ever have reached to the completion.

Aman Verma - EN22CS301110
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Anjali Jaiswal - EN22CS301132
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Ankit Parmar - EN22CS301135
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Ansh Laad - EN22CS301141
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Apoorva Gupta - EN22CS301190
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Arya Bhawsar - EN22CS301209
B.Tech. IV Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

Abstract

EduPlatform AI is an intelligent, cloud-based educational platform that integrates **Generative AI, multi-agent architecture, and modern DevOps practices** to deliver a fully automated and interactive learning experience. The system enables users to input a topic and select a desired difficulty level, upon which it dynamically generates a complete learning module consisting of a structured curriculum, detailed topic explanations, AI-generated visual content, assessment quizzes, and performance-based feedback.

The core of the platform is a **multi-agent pipeline**, where specialized agents—namely the Curriculum Agent, Tutor Agent, Quiz Agent, and Feedback Agent—collaborate in a sequential and context-aware manner to simulate a real-world teaching and evaluation process. The **Generative AI layer**, powered by large language models for text generation and image synthesis tools for visual content, ensures that the learning material is both comprehensive and multimodal, improving user engagement and conceptual understanding.

The system is implemented using an interactive web interface and is fully containerized using Docker for portability and consistency. It is deployed on cloud infrastructure with automated CI/CD pipelines, enabling continuous integration, efficient updates, and scalability. This combination of **GenAI capabilities, agent-based workflow orchestration, and cloud deployment** reflects a production-ready architecture suitable for real-world EdTech applications.

The primary objective of this project is to reduce manual effort in educational content creation, provide personalized and adaptive learning experiences, and demonstrate the practical integration of advanced AI technologies in education. Future enhancements include incorporating user progress tracking, adaptive learning mechanisms, retrieval-augmented generation (RAG), and scalable microservices architecture to further improve system intelligence and performance.

Keywords:

Generative AI,

Multi-Agent Systems, Intelligent Tutoring System, Multimodal Learning,

Cloud Computing,

Docker,

CI/CD,

AWS EC2,

Large Language Models (LLMs),

Content Generation,

AI-based Education,

Streamlit,

DevOps,

Adaptive Learning,

Quiz Generation,

Educational Technology (EdTech)

Table of Contents

		Page No.
	Report Approval	iii
	Declaration	iv
	Certificate	v
	Acknowledgement	vi-vii
	Abstract	viii
	Table of Contents	x-xi
	List of figures	xii
	List of tables	xii
	Abbreviations	xiii
	Notations & Symbols	xiii
Chapter 1	Introduction (Whichever is applicable)	1
	1.1 Introduction	1-2
	1.2 Literature Review	3
	1.3 Objectives	4
	1.4 Significance	5
	1.5 Research Design	6
	1.6 Source of Data	7
	1.7 Chapter Scheme	8-9
Chapter 2	Present Investigation	10
	2.1 Experimental Set-up	10-11
	2.2 Procedures Adopted	11-12
Chapter 3	System Design	13
	3.1 Application Design	13
	3.2 Process Flow.	14
	3.3 Information Flow.	14
	3.4 Components Design	15
Chapter 4	Implementation	20
	4.1 Development Environment	20-22
	4.2 Technology Stack	22
	4.3 Module-wise Implementation	23
	4.4 API Integration	24-25
Chapter 5	DevOps and Deployment	26
	5.1 Dockerization	26
	5.2 CI/CD Pipeline	26-29
	5.3 AWS EC2 Deployment	30
Chapter 6	Results and Discussions	31

	6.1 System Output	31-33
	6.2 Challenges Faced	34
Chapter 7	Summary and Conclusions	35
	7.1 Summary	35
	7.2 Conclusion	36
Chapter 8	Future scope	37
	Appendix	37-39
	Bibliography	39

List of Figures

3.1 Application Design	13
3.2 Process Flow.	14
3.3 Information Flow.	14
3.4 Components Design	15
3.5 Sequence Diagram	16
3.6 Deployment Diagram	17
3.7 System Architecture Diagram	18
3.8 Data Flow Diagram	19

List of Table

Table 1	Expermintal Setup (Compoent)	10
Table 2	Component Design	15
Table 3	Development Environment	21
Table 4	Docker Configuration Parameters	26

Abbreviations

AI – Artificial Intelligence

GenAI – Generative Artificial Intelligence

LLM – Large Language Model

MCQ – Multiple Choice Question

AWS – Amazon Web Services

EC2 – Elastic Compute Cloud

CI/CD – Continuous Integration / Continuous Deployment

EdTech – Educational Technology

Notations & Symbols

T – Input Topic

L – Learning Level (Beginner / Intermediate / Advanced)

C – Curriculum Output (Structured roadmap)

E – Explanation Content (Generated lesson)

V – Visual Content (AI-generated images)

Q – Quiz Set (MCQs)

A – User Answers

F – Feedback Output (Evaluation & explanation)

P – Pipeline Flow (Sequence of agents)

M – Model (LLM used for generation)

Chapter-1

1.1 Introduction

In recent years, the field of **Educational Technology (EdTech)** has evolved significantly with the integration of **Artificial Intelligence (AI)**. Traditional learning systems often rely on static content and lack personalization, making it difficult for learners to fully engage with complex topics. With the advancement of **Generative AI (GenAI)** and **Large Language Models (LLMs)**, it has become possible to automatically generate high-quality educational content tailored to user needs.

EduPlatform AI is designed to address these challenges by providing an intelligent, automated learning system that dynamically generates structured educational content. The platform allows users to input a topic and select a desired difficulty level, after which it produces a complete learning module including a curriculum roadmap, detailed explanations, visual aids, quizzes, and feedback. This eliminates the need for manual content creation and enables a more interactive learning experience.

The system leverages a **multi-agent architecture**, where specialized agents—such as Curriculum, Tutor, Quiz, and Feedback agents—collaborate to simulate a real-world teaching process. Each agent performs a specific task, contributing to a modular and scalable design. Additionally, the integration of **multimodal learning**, combining text and AI-generated images, enhances understanding and engagement.

From a system perspective, the platform is developed using a web-based interface and deployed on cloud infrastructure using **Docker and AWS EC2**, with automated **CI/CD pipelines** ensuring continuous integration and deployment. This demonstrates a production-ready approach by combining **GenAI, agent-based workflow, and DevOps practices** into a unified solution.

The primary goal of this project is to create an intelligent tutoring system that improves learning efficiency, reduces dependency on static educational resources, and showcases the practical application of advanced AI technologies in real-world scenarios.

EduPlatform AI is proposed as an intelligent, automated learning system that leverages advanced AI techniques to generate end-to-end educational content tailored to user input. The platform accepts a topic and a specified difficulty level and utilizes a combination of LLM-based text generation and AI-driven image synthesis to produce a comprehensive learning module. This module includes a structured curriculum roadmap, in-depth conceptual explanations, context-aware visual representations, assessment questions, and performance-based feedback. By automating the entire content generation pipeline, the system minimizes reliance on manually curated resources and enables scalable content delivery across multiple domains and difficulty levels.

A key aspect of the proposed system is its multi-agent architecture, which decomposes the overall learning workflow into specialized, task-oriented components. The Curriculum Agent is responsible for generating a logically ordered learning path, ensuring that concepts are introduced in a progressive and coherent manner. The Tutor Agent utilizes LLM capabilities to produce detailed explanations with contextual relevance and clarity. The Quiz Agent generates assessment

items designed to evaluate conceptual understanding, while the Feedback Agent analyzes user responses to provide adaptive evaluation and improvement suggestions. This agent-based design promotes modularity, reusability, and scalability, allowing individual components to be enhanced or replaced without affecting the overall system.

Furthermore, the platform incorporates a multimodal learning approach by integrating textual content with AI-generated visual aids. The inclusion of visual representations enhances cognitive understanding by presenting information in multiple formats, which is particularly beneficial for complex or abstract topics. The GenAI layer, powered by high-performance inference models, ensures efficient and context-aware content generation, while external image generation services contribute to the creation of relevant visual materials without requiring extensive manual design efforts.

From a system implementation perspective, EduPlatform AI is developed as a web-based application using a lightweight and interactive user interface framework, enabling real-time user interaction and content visualization. The backend architecture is designed with a clear separation of concerns between the AI generation layer and the agent orchestration pipeline, ensuring maintainability and extensibility. The entire application is containerized using Docker, which guarantees consistency across development and production environments. Deployment is carried out on cloud infrastructure, specifically virtual compute instances, providing scalability and availability for multiple concurrent users. Additionally, the integration of Continuous Integration and Continuous Deployment (CI/CD) pipelines automates the build, testing, and deployment processes, ensuring rapid iteration and reliable software delivery.

The system also reflects principles of cloud-native and production-ready design by combining AI-driven computation with DevOps practices. The use of containerization, environment-based configuration management, and automated deployment workflows demonstrates a practical approach to deploying AI applications in real-world scenarios. Moreover, the modular agent-based framework enables future extensions such as user personalization, learning analytics, voice-based interaction, and retrieval-augmented generation (RAG) for document-based learning, thereby increasing the scope and applicability of the platform.

The primary objective of this project is to design and implement an intelligent tutoring system that enhances learning efficiency through automation, personalization, and interactivity. By integrating Generative AI, multi-agent systems, and cloud deployment technologies, EduPlatform AI aims to provide a scalable and adaptive educational solution that reduces dependency on static content, improves learner engagement, and demonstrates the practical application of advanced AI methodologies in the domain of digital education.

1.2 Literature Review

The development of **EduPlatform AI** is influenced by advancements in multiple domains, including Generative AI, Intelligent Tutoring Systems, Multi-Agent Systems, and Cloud-based deployment architectures.

Early **Intelligent Tutoring Systems (ITS)** focused on rule-based approaches to provide personalized learning experiences. While effective in limited scenarios, these systems lacked scalability and adaptability. With the emergence of **Natural Language Processing (NLP)** and **Large Language Models (LLMs)**, modern systems are now capable of generating human-like explanations and understanding user queries more effectively.

Recent research in **Generative AI** has demonstrated the capability of models to automatically generate educational content such as summaries, explanations, and assessment questions. These models significantly reduce the effort required for content creation and enable dynamic learning environments. Additionally, the integration of **multimodal learning**, which combines text and visual elements, has been shown to improve knowledge retention and conceptual clarity.

The concept of **Multi-Agent Systems** has gained attention for solving complex problems by dividing tasks among specialized agents. In educational platforms, agent-based architectures allow for modular design, where each agent can handle a specific function such as content generation, evaluation, or feedback. This approach improves system flexibility and maintainability.

Despite these advancements, many existing educational platforms still lack full integration of **GenAI, agent-based workflows, and cloud deployment** into a single system. **EduPlatform AI** addresses this gap by combining these technologies into a unified platform, providing an end-to-end automated learning solution that is scalable, interactive, and production-ready.

In addition to these core areas, recent developments in adaptive learning systems emphasize the importance of personalization through data-driven approaches. Modern platforms increasingly utilize learner interaction data, behavioral patterns, and performance metrics to dynamically adjust content difficulty and presentation style. Such adaptive mechanisms contribute to improved learner engagement and more efficient knowledge acquisition. However, many existing systems rely heavily on predefined datasets and lack real-time generative capabilities, limiting their responsiveness to new or diverse topics.

In summary, existing research highlights significant progress in individual domains such as AI-based content generation, intelligent tutoring, and scalable system deployment. However, there is a clear gap in integrating these components into a cohesive, automated, and production-ready educational platform. EduPlatform AI contributes to this space by unifying Generative AI, multi-agent system design, multimodal learning, and cloud-based DevOps practices into a single architecture, thereby advancing the development of next-generation intelligent educational systems.

1.3 Objectives

* To design and develop an AI-based educational platform capable of automatically generating structured and comprehensive learning content from a given topic and user-defined difficulty level.

* To implement advanced Generative AI (GenAI) techniques and Large Language Models (LLMs) for dynamic, context-aware content generation.

* To build a multi-agent system architecture consisting of Curriculum, Tutor, Quiz, and Feedback agents to ensure modularity, scalability, and efficient task distribution.

* To design an agent orchestration pipeline that enables seamless communication and coordination between different agents in the learning workflow.

* To generate a well-structured curriculum roadmap that organizes topics in a logical and progressive manner for effective learning.

* To develop a Tutor Agent that produces detailed, accurate, and easy-to-understand explanationstailored to the learner's level.

* To implement a Quiz Agent capable of generating dynamic and diverse assessment questions for evaluating learner understanding.

* To design a Feedback Agent that analyzes user responses and provides adaptive feedback, performance evaluation, and improvement suggestions.

* To incorporate multimodal learning by integrating AI-generated visual content along with textual explanations to enhance comprehension and retention.

* To create an interactive user interface that allows real-time content generation, user input handling, and seamless learning experience.

* To ensure scalability and portability of the application through containerization using Docker technology.

* To deploy the system on cloud infrastructure (AWS EC2) for high availability, reliability, and real-world accessibility.

* To implement Continuous Integration and Continuous Deployment (CI/CD) pipelines for automated building, testing, and deployment of the application.

* To design the system in a modular and extensible manner, enabling future enhancements such as user authentication, progress tracking, and advanced analytics.

* To demonstrate the practical integration of AI, agent-based systems, and DevOps practices in building a production-ready intelligent tutoring platform.

1.4 Significance

- Reduces manual effort in creating educational content by automating the entire learning pipeline.
- Enhances learning experience through personalized and interactive AI-generated content.
- Demonstrates practical integration of Generative AI, multi-agent systems, and DevOps in a real-world application.
- Provides a scalable and cloud-deployed solution suitable for modern EdTech platforms.
- Serves as a foundation for future improvements like adaptive learning and intelligent tutoring systems.

1.4.1 Technological Significance

EduPlatform AI demonstrates the practical viability of integrating cutting-edge AI technologies — LLMs, image generation models, and multi-agent architectures — into a production-grade application. It serves as a reference implementation for developers and researchers interested in building similar systems, providing insights into architecture design, API integration, and deployment strategies.

1.4.2 Educational Significance

By automating the content creation process and delivering personalized, multimodal learning experiences, EduPlatform AI has the potential to democratize access to high-quality educational content. It reduces the dependency on domain experts for content creation and enables learners to access customized learning modules on demand, at any difficulty level.

1.4.3 Practical Significance

From a practical standpoint, EduPlatform AI demonstrates the applicability of DevOps practices — containerization, cloud deployment, and CI/CD — to AI-powered applications. This integration is increasingly important in the industry, where the ability to rapidly develop, test, and deploy AI systems is a critical competitive advantage.

1.5 Research Design

The research design for **EduPlatform AI** follows a **system development and experimental approach**, focusing on the integration of **Generative AI, multi-agent architecture, and cloud deployment**. The system is designed by decomposing the learning process into multiple stages, handled by specialized agents—Curriculum, Tutor, Quiz, and Feedback—to simulate an end-to-end educational workflow.

The research design for EduPlatform AI follows a system development and experimental approach, The overall design adopts a **modular and iterative development methodology**, where each component of the system is independently developed, tested, and integrated into the complete pipeline.

This modular approach ensures flexibility, maintainability, and scalability, allowing improvements or modifications in individual agents without affecting the entire system.

The research emphasizes the practical implementation of AI-driven educational systems rather than purely theoretical modeling, thereby aligning with real-world application requirements.

The system architecture is divided into multiple layers, including the **user interaction layer, AI generation layer, agent orchestration layer, and deployment layer**. The user interaction layer captures input parameters such as topic and difficulty level through an interactive web interface.

The AI generation layer leverages Large Language Models (LLMs) for text generation and external image generation tools for producing visual content.

The agent orchestration layer coordinates the execution of different agents in a sequential and logical manner, ensuring that the output of one agent serves as the input for the next stage. Finally, the deployment layer ensures accessibility and scalability through cloud-based infrastructure.

From an experimental perspective, the system is evaluated based on its ability to generate coherent, relevant, and structured educational content.

The evaluation criteria include **content quality, logical flow of curriculum, accuracy of explanations, relevance of generated questions, and effectiveness of feedback mechanisms**. The research also considers user interaction aspects such as response time, usability, and overall learning experience.

Although formal quantitative metrics may be limited, qualitative analysis is performed by testing the system across different topics and difficulty levels to assess consistency and adaptability.

The implementation process involves integrating multiple technologies, including Generative AI models for content creation, a multi-agent framework for workflow management, and web technologies for user interaction.

The system is containerized using Docker to ensure consistent execution across different environments, and it is deployed on cloud infrastructure to support real-time usage.

Continuous Integration and Continuous Deployment (CI/CD) pipelines are incorporated to automate the build and deployment processes, reflecting modern DevOps practices in AI system development.

Additionally, the research design considers extensibility and future enhancements as part of its framework. The architecture is intentionally designed to support additional features such as user personalization, progress tracking, and integration with external knowledge sources using retrieval-based techniques.

This forward-looking design approach ensures that the system can evolve with advancements in AI technologies and changing educational requirements.

1.6 Source of Data

The data used in EduPlatform AI is primarily generated dynamically using Generative AI models rather than relying on a fixed dataset. T

he system utilizes Large Language Models (LLMs) to produce structured educational content, explanations, quiz questions, and feedback based on user-provided topics and difficulty levels.

For visual content, AI-based image generation tools are used to create relevant diagrams and illustrations. Additionally, the system may rely on publicly available knowledge learned by pre-trained models, which are trained on large-scale text corpora.

1.7 Chapter Scheme

The report is organized into different chapters, each focusing on a specific aspect of the EduPlatform AI system.

- **Chapter 1: Introduction**
  This chapter presents the overview of the project, including introduction, literature review, objectives, significance, research design, and source of data.

- **Chapter 2: System Analysis**
  This chapter explains the problem statement, existing system, proposed system, system requirements, and feasibility analysis.

- **Chapter 3: System Design**
  This chapter describes the architecture and design of the system, including the GenAI layer, multi-agent pipeline, and user interface design.

- **Chapter 4: Implementation**
  This chapter details the technologies and tools used for development, such as Streamlit, Groq, Docker, and deployment on **Amazon Web Services EC2, along with module-wise implementation.

- **Chapter 5: Testing and Results**
  This chapter includes system testing, test cases, validation, and the results obtained from the application.

- **Chapter 6: DevOps and Deployment**
  This chapter explains containerization using Docker, CI/CD pipeline implementation using GitHub Actions, and deployment on AWS EC2.

- **Chapter 7: Conclusion and Future Scope**
  This chapter summarizes the overall work and suggests future enhancements such as user authentication, progress tracking, and scalability improvements.

Chapter 2

Present Investigation

2.1 Experimental Set-up

The experimental setup for the *EduPlatform AI* system consists of a cloud-based environment where the application is developed, tested, and deployed. The system is built using Python and implemented through **Streamlit** for the user interface.

The backend integrates the **Groq** API for text generation and Pollinations AI for image generation. A multi-agent architecture is used to process user input through different stages such as curriculum generation, tutoring, quiz creation, and feedback evaluation.

For deployment, the application is containerized using Docker and hosted on an Amazon Web Services EC2 instance. Continuous Integration and Continuous Deployment (CI/CD) is implemented using **GitHub Actions**, enabling automated build and deployment of the system.

This setup ensures scalability, reproducibility, and efficient execution of the AI-based learning platform.

Component	Specification
Operating System	Ubuntu 22.04 LTS (on AWS EC2)
Instance Type	AWS EC2 t2.micro
Programming Language	Python 3.10
Web Framework	Streamlit 1.32.0
LLM API	Groq API (llama3-70b-8192)
Image Generation	Pollinations AI
Containerization	Docker 24.0
CI/CD	GitHub Actions
Version Control	Git & GitHub

Table 1

In addition to the core setup, the development environment is configured with all required dependencies specified in a requirements file, ensuring consistency across local and cloud-based executions. Environment variables are managed securely using configuration files to handle sensitive data such as API keys. The system is tested in both local and remote environments to validate functionality, compatibility, and performance before deployment.

The CI/CD pipeline automates various stages of the software lifecycle, including code integration, image building, and deployment to the cloud server.

This not only reduces manual intervention but also ensures faster updates and continuous improvement of the system. Version control is maintained using a Git-based repository, allowing systematic tracking of changes and collaboration.

Furthermore, the system is evaluated under different input conditions, including varying topics and difficulty levels, to test its robustness and adaptability. Performance aspects such as response time, system stability, and accuracy of generated outputs are observed during experimentation.

This comprehensive setup ensures that the platform operates reliably under real-world conditions and can be scaled for broader usage.

Overall, the experimental setup is designed to reflect a real-world deployment scenario, combining AI model integration, agent-based workflow execution, and cloud-based infrastructure to deliver a fully functional and production-ready intelligent learning system.

2.2 Procedures Adopted

The development of *EduPlatform AI* followed a systematic procedure starting with requirement analysis and system design. Initially, the architecture was defined by integrating a GenAI layer with a multi-agent pipeline to handle different stages of content generation.

The application was then developed using Python with Streamlit for the user interface and Groq for generating textual content, along with Pollinations AI for visual content. Individual agents were implemented for curriculum generation, tutoring, quiz creation, and feedback evaluation, and were orchestrated through a pipeline.

After development, the system was containerized using Docker to ensure consistency across environments. Finally, the application was deployed on an **Amazon Web Services EC2 instance, with CI/CD automation handled through GitHub Actions for continuous integration and deployment.

This structured approach ensured efficient development, testing, and deployment of the platform.

In addition to the core procedure, the development process began with a detailed **requirement analysis phase**, where system objectives, user needs, and functional specifications were identified. This was followed by a **system design phase**, in which the overall architecture, data flow, and interaction between different components were clearly defined. Special emphasis was placed on designing a modular multi-agent framework to allow independent development and scalability of each component.

During the implementation phase, each agent was developed and tested individually to ensure correctness and performance before integration. The Curriculum Agent was designed to generate logically structured learning paths, while the Tutor Agent focused on producing clear and context-aware explanations. The Quiz and Feedback agents were implemented to support assessment and evaluation functionalities. These agents were then integrated using a pipeline-based orchestration mechanism, enabling smooth data flow and coordination between components.

A dedicated **testing phase** was conducted to validate system functionality, including unit testing of individual modules and integration testing of the complete pipeline. The system was tested with various topics and difficulty levels to evaluate its adaptability, accuracy, and consistency in content generation. User interaction scenarios were also simulated to assess usability and response behavior.

Following successful testing, the system was prepared for deployment through containerization. Docker was used to package the application along with all dependencies, ensuring portability and eliminating environment-related issues. The deployment process involved configuring the cloud server, running container instances, and exposing necessary ports for user access.

The integration of CI/CD pipelines automated the entire workflow from code updates to deployment. Each code change triggered automated build and deployment processes, ensuring that the latest version of the application was consistently available without manual intervention. This approach improved development efficiency and reduced deployment time.

Furthermore, the procedure included monitoring and iterative improvement, where system performance and output quality were continuously observed and refined. Feedback from testing phases was used to enhance prompt design, agent coordination, and overall system efficiency.

Overall, the adopted procedure reflects a well-structured software engineering lifecycle, combining requirement analysis, modular development, rigorous testing, and automated deployment to build a robust and scalable AI-powered educational platform.

Chapter 3

System Design

3.1 Application Design

The application follows a layered architecture consisting of a user interface built with Streamlit, a GenAI processing layer using Groq, and a multi-agent pipeline. The system is containerized using Docker and deployed on Amazon Web Services EC2.

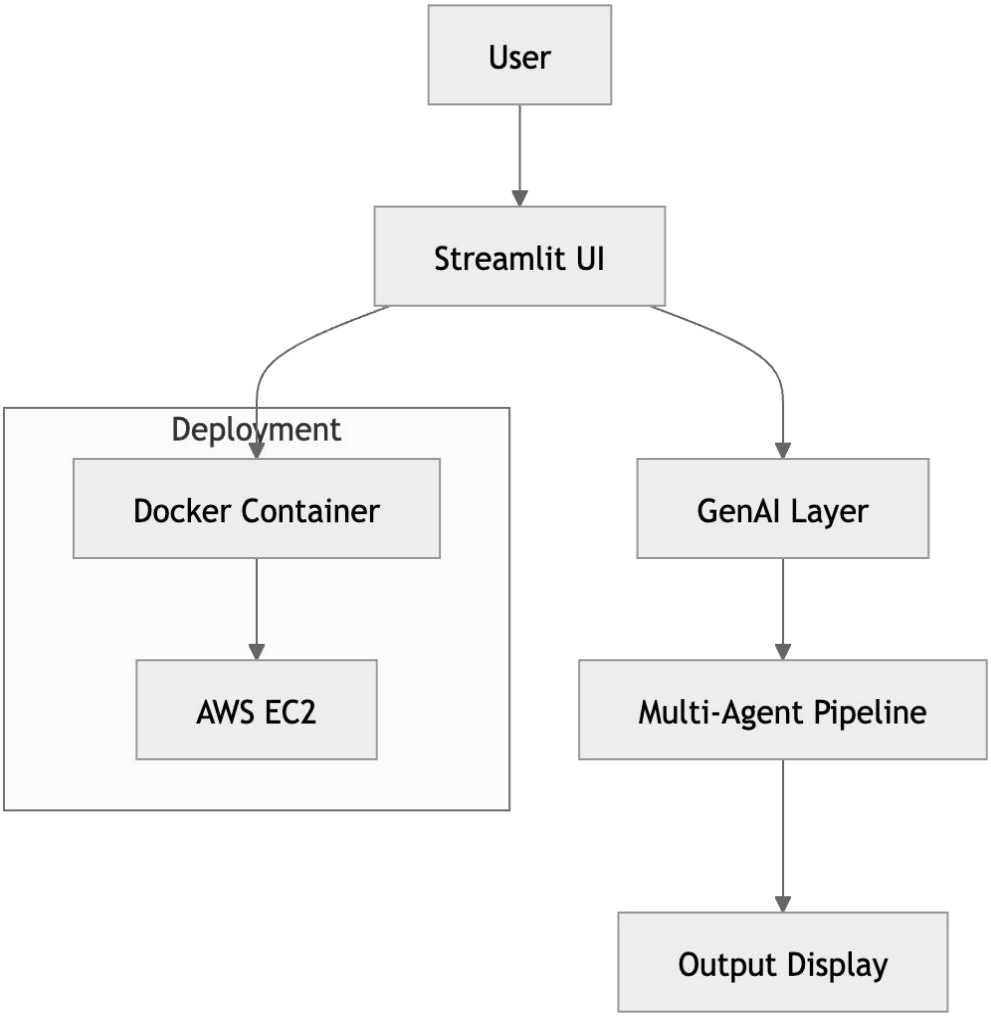


Fig 3.1 Application Design

3.2 Process Flow

The process begins with user input, followed by sequential execution of agents: curriculum, tutor, quiz, and feedback. Each stage refines the output.

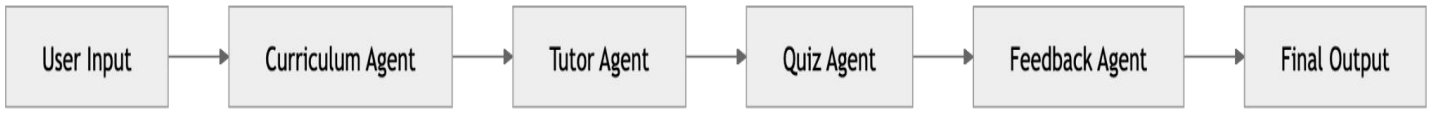


Fig 3.2 Process Flow

3.3 Information Flow

The information flow in the EduPlatform AI system begins with the user providing input through the **Streamlit** interface, which is then passed to the GenAI layer powered by **Groq**. The processed data is forwarded to the multi-agent pipeline, where each agent (curriculum, tutor, quiz, and feedback) sequentially refines and enriches the content. The final structured output, including explanations, visuals, and assessments, is then sent back to the user interface for display.

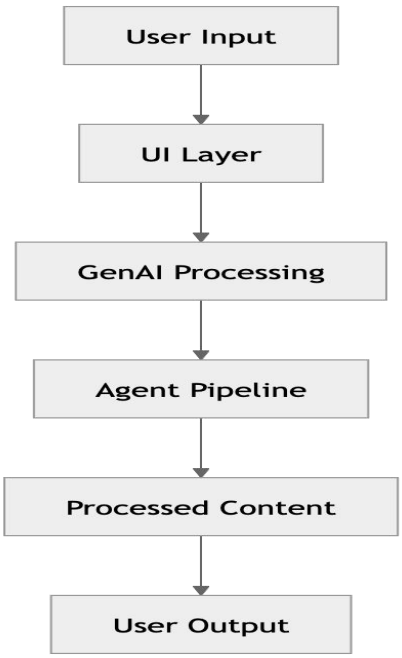


Fig 3.3 Information Flow

3.4 Components Design

The EduPlatform AI system is composed of multiple interconnected components, including the user interface built with Streamlit, the GenAI module using Groq for text generation and an image generation module, and a set of intelligent agents responsible for curriculum creation, tutoring, quiz generation, and feedback evaluation. These components are orchestrated through a central pipeline controller, while the entire application is containerized using Docker and deployed on Amazon Web Services EC2 to ensure scalability and reliability.

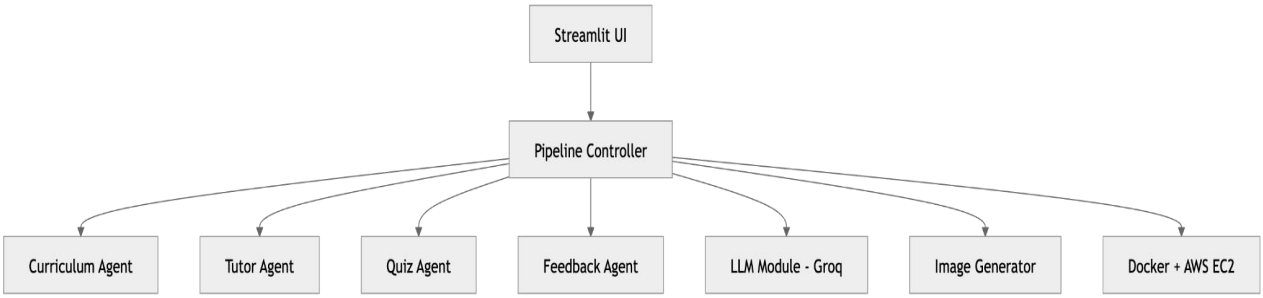
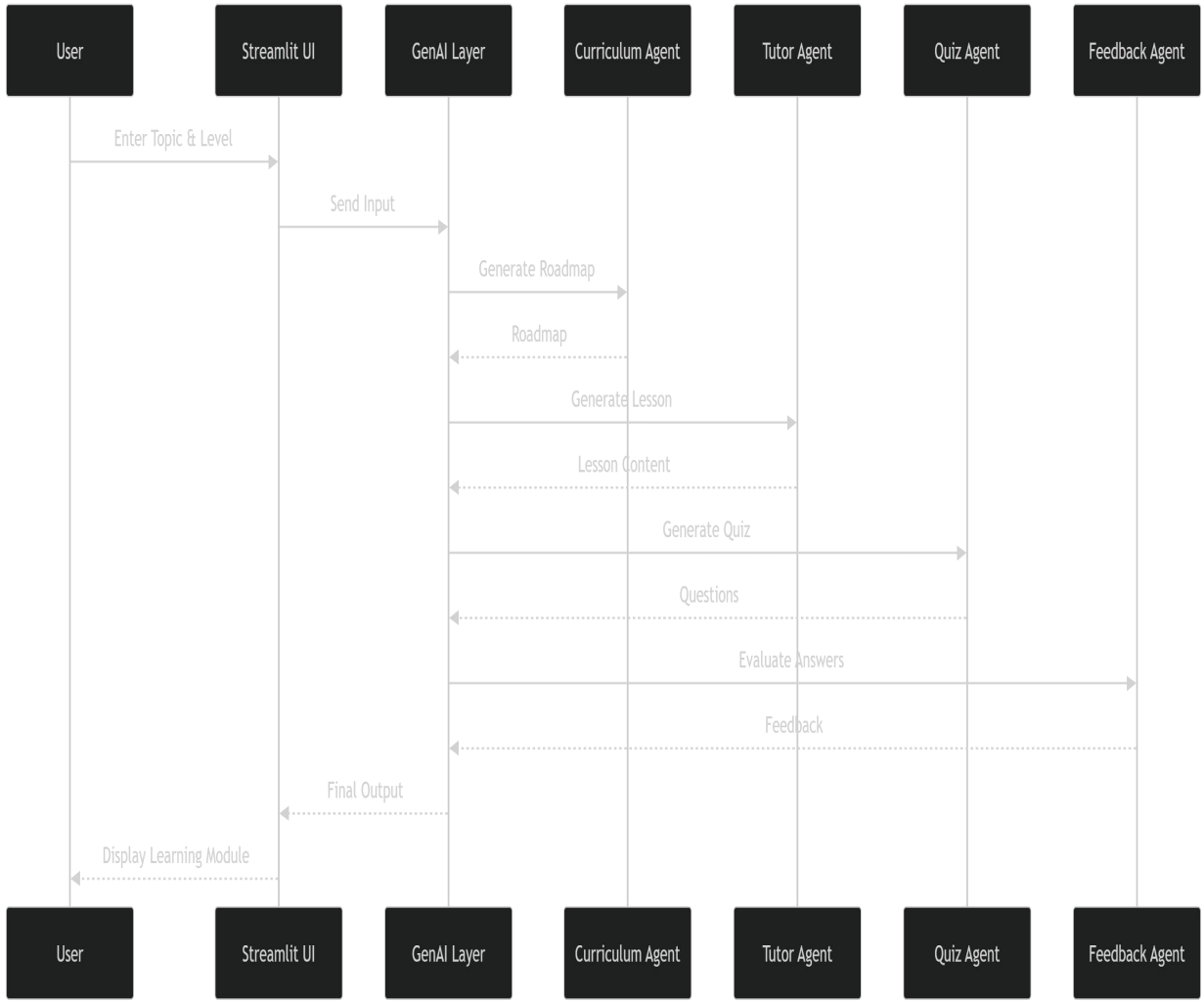


Fig 3.4 Component Design

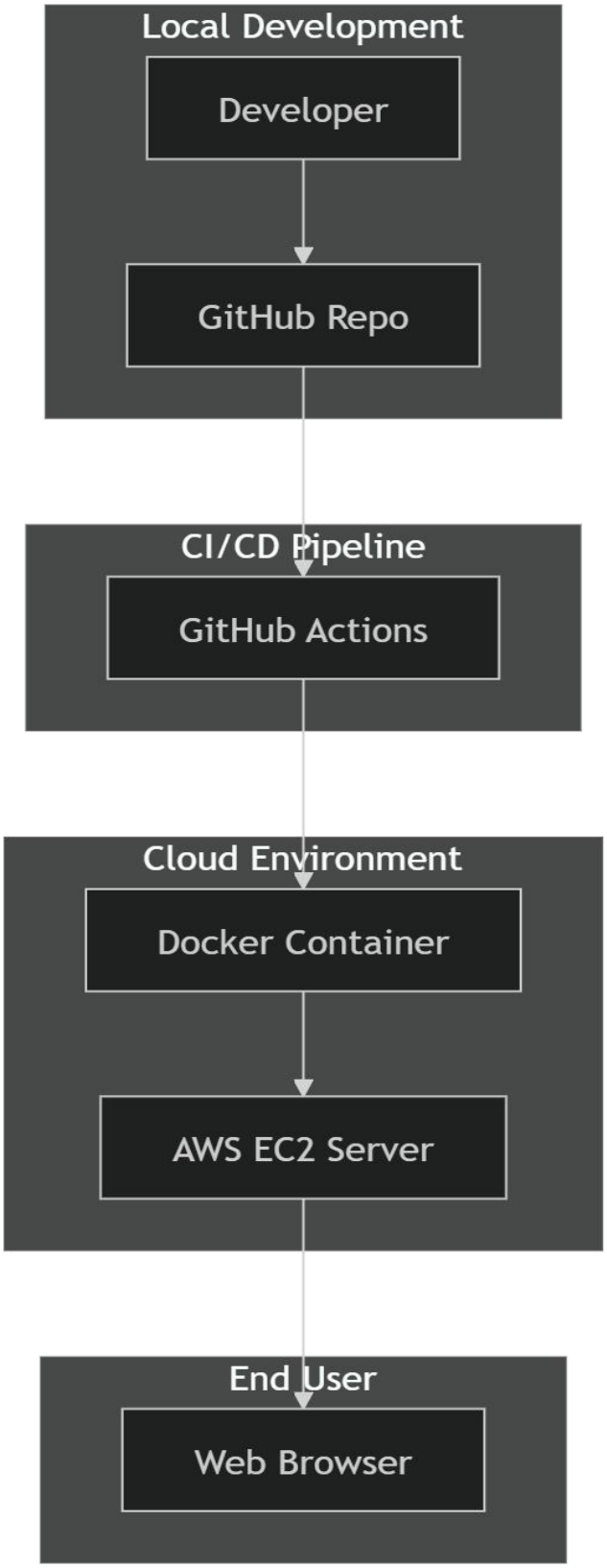
Component	Technology	Responsibility
User Interface	Streamlit	Input capture, output display, navigation
Pipeline Controller	Python	Agent orchestration, state management, error handling
Curriculum Agent	Python + Groq API	Learning roadmap and subtopic generation
Tutor Agent	Python + Groq API	Detailed topic explanations and examples
Quiz Agent	Python + Groq API	MCQ generation based on learning content
Feedback Agent	Python + Groq API	Answer evaluation and corrective feedback
Image Generator	Pollinations AI	Topic-relevant visual content synthesis
Docker Container	Docker	Application packaging and environment isolation
CI/CD Pipeline	GitHub Actions	Automated build, test, and deployment
Cloud Host	AWS EC2	Application hosting and public accessibility

Table 2

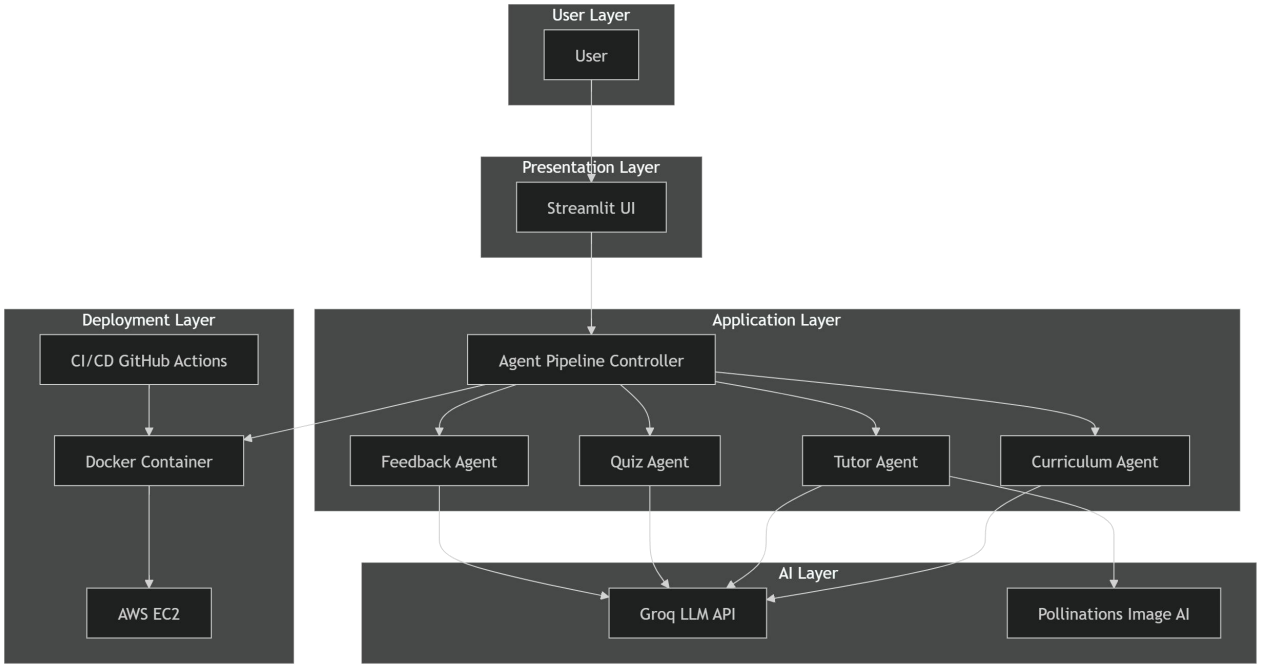
3.5 Sequence Diagram



3.6 Deployment Diagram

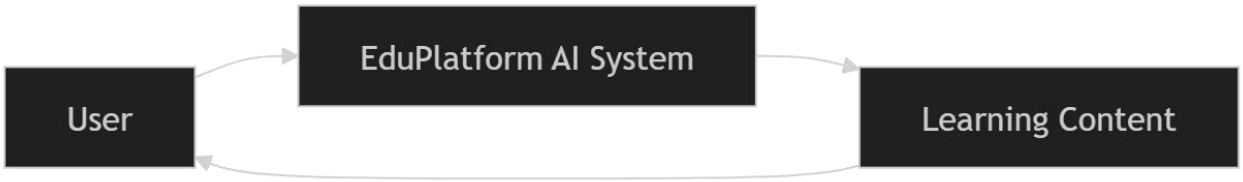


3.7 System Architecture Diagram

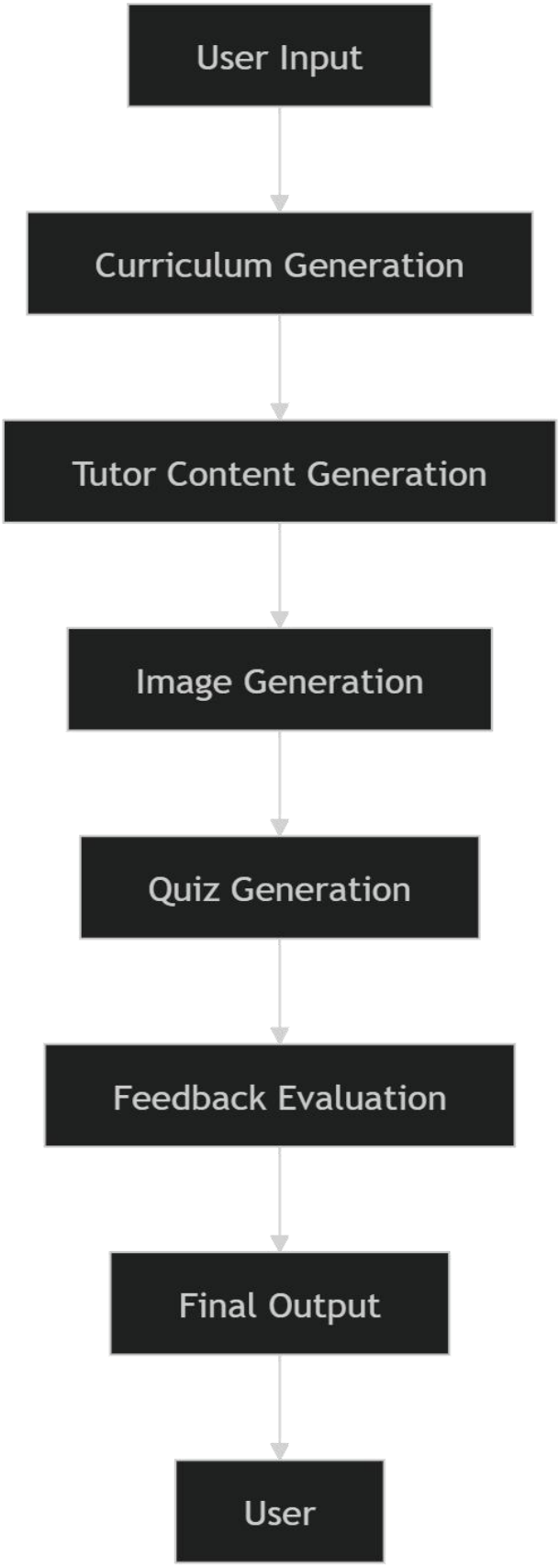


3.8 Data Flow Diagram

3.8.1 Data Flow Diagram (DFD - Level 0)



3.8.2 Data Flow Diagram (DFD - Level 1)



Chapter 4

Implementation

4.1 Development Environment

The EduPlatform AI system was developed using a modern and efficient development environment to ensure smooth implementation and deployment. The primary programming language used is Python, due to its extensive support for AI and web-based applications. The user interface is built using Streamlit, which allows rapid development of interactive applications.

The backend integrates the Groq API for generating educational content, along with Pollinations AI for image generation. The development process is managed using Git for version control and GitHub for repository hosting.

For deployment and environment consistency, Docker is used to containerize the application. The final application is hosted on an Amazon Web Services EC2 instance, ensuring accessibility and scalability.

Continuous integration and deployment are handled using GitHub Actions, enabling automated build and deployment workflows.

This development environment ensures reliability, scalability, and efficient management of the AI-based educational platform.

In addition to the core technologies, the development environment is structured to support **modularity, maintainability, and rapid iteration**. The project follows a well-organized directory structure separating core AI logic, agent modules, and user interface components, which simplifies development and debugging.

Python virtual environments are used during local development to manage dependencies effectively and avoid conflicts between packages.

The application leverages various Python libraries and frameworks to support functionality such as API integration, data handling, and asynchronous processing. Environment variables are managed through configuration files to securely store sensitive information like API keys, ensuring better security practices during development and deployment.

Tool / Technology	Version	Purpose
Python	3.10.12	Primary programming language
Streamlit	1.32.0	Web UI framework
Groq SDK	0.6.0	LLM API client
Requests	2.31.0	HTTP client for Pollinations AI
Docker	24.0.5	Containerization
Docker Compose	2.20.2	Multi-container orchestration
Git	2.42.0	Version control
GitHub Actions	N/A (cloud)	CI/CD pipeline
VS Code	1.85.0	Integrated Development Environment
Pytest	7.4.3	Unit and integration testing
python-dotenv	1.0.0	Environment variable management

Table 3

The Streamlit-based frontend provides a lightweight yet powerful interface that supports real-time interaction, dynamic content rendering, and easy integration with backend services. This enables developers to quickly prototype and test features without requiring complex frontend frameworks.

From a backend perspective, the integration of Groq API enables high-performance inference for text generation, while Pollinations AI provides a simple and efficient solution for generating relevant visual content. These integrations are designed to work seamlessly within the agent-based pipeline, ensuring smooth data flow and minimal latency.

The use of Docker plays a critical role in maintaining environment consistency across different stages of development, testing, and deployment. By packaging the application along with all dependencies into a container, it eliminates issues related to system compatibility and simplifies the deployment process. Docker also supports scalability by allowing multiple instances of the application to run concurrently if required.

The cloud environment, based on AWS EC2, is configured to host the containerized application and handle incoming user requests. It provides flexibility in resource allocation, enabling the

system to scale based on demand. Proper network and security configurations are applied to ensure secure and stable access to the platform.

Furthermore, the integration of GitHub Actions for CI/CD automates the entire software delivery pipeline. Code changes pushed to the repository trigger automated workflows that build the Docker image, test the application, and deploy it to the cloud server. This reduces manual effort, minimizes errors, and ensures continuous availability of updated features.

Overall, the development environment is designed to align with modern software engineering and DevOps practices, enabling efficient development, seamless integration of AI components, and reliable deployment of a scalable, production-ready educational platform.

4.2 Technology Stack

- **Programming Language:** Python
- **Frontend/UI:** Streamlit
- **AI/LLM:** Groq
- **Image Generation:** Pollinations AI
- **Architecture:** Multi-Agent System (Curriculum, Tutor, Quiz, Feedback)
- **Backend Logic:** Python-based pipeline orchestration
- **Containerization:** Docker
- **Cloud Platform:** Amazon Web Services EC2
- **CI/CD:** GitHub Actions
- **Version Control:** Git & GitHub

4.3 Module-wise Implementation

The EduPlatform AI system is divided into multiple modules to ensure scalability, maintainability, and efficient processing of tasks.

1. User Interface Module

The user interface is developed using **Streamlit**, which allows users to input the topic and difficulty level. It also displays the generated learning content, including explanations, images, quizzes, and feedback in an interactive format.

2. GenAI Module

This module handles AI-based content generation. It integrates the **Groq** API to generate structured educational text such as lessons and explanations. Additionally, it uses Pollinations AI for generating visual learning content.

3. Multi-Agent Module

The system implements a multi-agent architecture where different agents perform specific tasks. The Curriculum Agent generates a structured roadmap, the Tutor Agent provides detailed explanations, the Quiz Agent creates questions, and the Feedback Agent evaluates user responses.

4. Pipeline Orchestration Module

This module controls the flow of execution between different agents. It ensures that the output from one agent is passed as input to the next, maintaining a sequential workflow and proper data flow throughout the system.

5. Deployment Module

The deployment module handles containerization and hosting of the application. Docker is used to

package the application into containers, and it is deployed on an **Amazon Web Services EC2 instance. CI/CD is implemented using **GitHub Actions** to automate the build and deployment process.

4.4 API Integration

The EduPlatform AI system integrates external APIs to enable dynamic and intelligent content generation. The primary API used in the system is the Groq API, which is responsible for generating textual educational content such as structured lessons, explanations, and quiz questions. The API is accessed through secure environment variables, ensuring safe handling of sensitive credentials.

In addition to text generation, the system also integrates Pollinations AI for generating visual learning content. This API does not require an API key and is used to create images based on the input topic, enhancing the overall learning experience.

The API calls are managed within the GenAI module, where user input is processed and sent to the respective APIs. The responses received are then formatted and passed through the multi-agent pipeline for further processing. This integration enables real-time, scalable, and automated content generation within the platform.

4.4.1 Groq API Integration

The Groq API is accessed via the official Groq Python SDK. Authentication is performed using an API key stored in a .env file and injected into the Docker container as an environment variable. The API is invoked using the chat.

completions.create endpoint, passing a list of message objects comprising a system prompt (agent role and instructions) and a user message (content generation request).

Responses are extracted from the choices[0].message.content field. The integration implements streaming mode for long-form content generation, enabling progressive display of content in the Streamlit UI.

4.4.2 Pollinations AI Integration

Pollinations AI is integrated via simple HTTP GET requests. The image prompt is URL-encoded and appended to the Pollinations API endpoint URL. The API returns a binary JPEG image, which is displayed in the Streamlit UI using st.image(). The module caches generated images by prompt hash to avoid redundant API calls within a session.

Chapter 5

DevOps and Deployment

5.1 Dockerization

Docker containerization is a cornerstone of the EduPlatform AI deployment strategy. The application is packaged into a Docker container to ensure environment consistency, portability, and ease of deployment across different hosting environments.

5.1.1 Dockerfile

The Dockerfile is based on the official python:3.10-slim image, which provides a minimal Python environment. The following steps are performed in the Dockerfile: (i) Setting the working directory to /app; (ii) Copying the requirements.txt file and installing dependencies using pip; (iii) Copying the application source code; (iv) Exposing port 8501 (Streamlit's default port); and (v) Setting the entrypoint to launch the Streamlit application. A .dockerignore file excludes development-only files (e.g., __pycache__, .env, .git) from the container image.

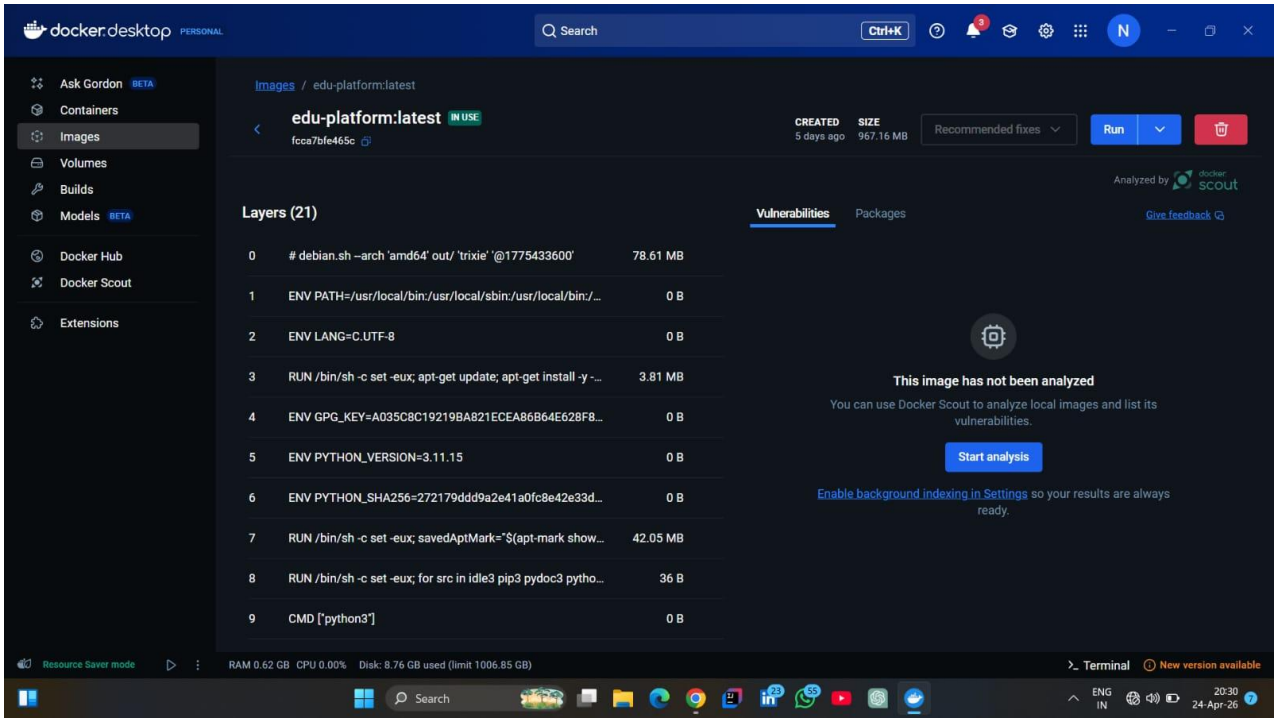
5.1.2 Docker Compose

A docker-compose.yml file is provided for local development and testing. It defines a single service (eduplatform) that builds from the local Dockerfile, maps port 8501 on the host to port 8501 in the container, and injects environment variables (including the Groq API key) from a .env file. This enables developers to run the full application locally with a single docker compose up command.

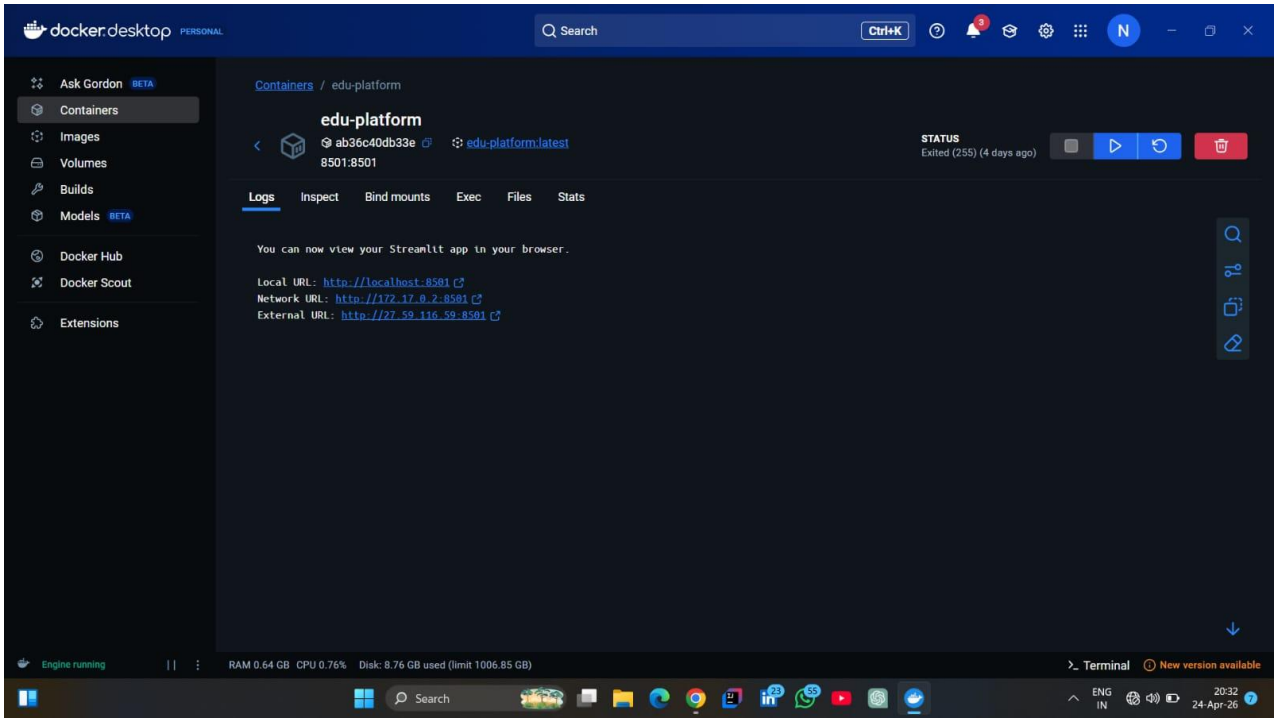
Docker Configuration	Value
Base Image	python:3.10-slim
Container Port	8501
Host Port Mapping	8501:8501
Environment Variables	GROQ_API_KEY (injected at runtime)
Container User	Non-root (appuser)
Restart Policy	unless-stopped
Image Size	~350 MB

Table 4

Image:



Containers:



5.2 CI/CD Pipeline

The CI/CD pipeline is implemented using GitHub Actions and is configured to trigger on every push to the main branch and on pull request events targeting the main branch. The pipeline comprises three primary stages:

5.2.1 Build Stage

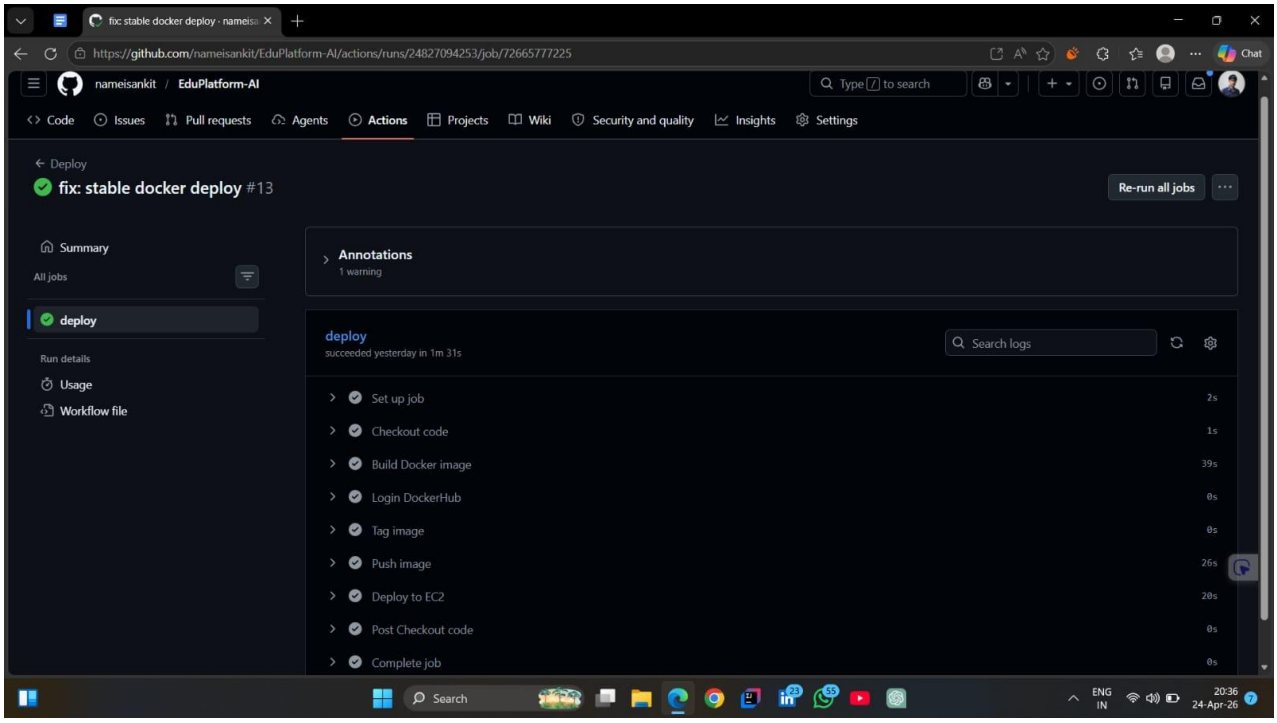
The Build stage checks out the repository code, sets up a Python 3.10 environment, installs all dependencies from requirements.txt, and runs the unit and integration test suite using pytest. If any test fails, the pipeline halts and reports the failure, preventing broken code from being deployed. A code quality check using flake8 is also performed to enforce coding standards.

5.2.2 Docker Build and Push Stage

Upon successful completion of the Build stage, the Docker Build and Push stage builds the Docker image using the repository's Dockerfile and pushes it to Docker Hub. The image is tagged with both the latest tag and a version tag derived from the Git commit SHA, enabling rollback to previous versions if needed. Docker Hub credentials are stored as GitHub Actions secrets and injected securely at build time.

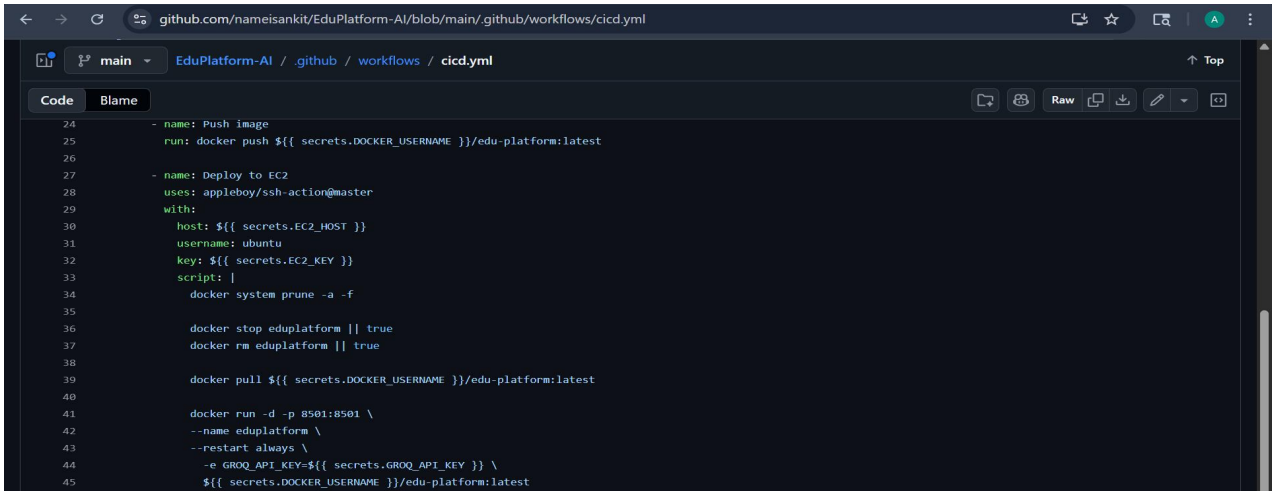
5.2.3 Deployment Stage

The Deployment stage establishes an SSH connection to the AWS EC2 instance and executes a deployment script that: (i) Pulls the latest Docker image from Docker Hub; (ii) Stops and removes the currently running container; and (iii) Starts a new container using the updated image, injecting the Groq API key as an environment variable. The deployment script implements a health check that verifies the new container is responding on port 8501 before declaring the deployment successful.


fix: stable docker deploy - nameisa
https://github.com/nameisankit/EduPlatform-AI/actions/runs/24827094253/job/72665777225
nameisankit / EduPlatform-AI
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Deploy
fix: stable docker deploy #13
Re-run all jobs
Summary
All jobs
deploy
Run details
Usage
Workflow file
Annotations
1 warning
deploy
succeeded yesterday in 1m 31s
Search logs
Set up job 2s
Checkout code 1s
Build Docker image 39s
Login DockerHub 0s
Tag image 0s
Push image 26s
Deploy to EC2 20s
Post Checkout code 0s
Complete job 0s

Workflow

github.com/nameisankit/EduPlatform-AI/blob/main/.github/workflows/cicd.yml
nameisankit fix: stable docker deploy
861f47a · 4 days ago
History
Code
Blame
Raw
name: Deploy
on:
push:
branches: [main]
jobs:
deploy:
runs-on: ubuntu-latest
steps:
- name: Checkout code
uses: actions/checkout@v3
- name: Build Docker image
run: docker build -t edu-platform .
- name: Login DockerHub
run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --password-stdin
- name: Tag image
run: docker tag edu-platform ${{ secrets.DOCKER_USERNAME }}/edu-platform:latest

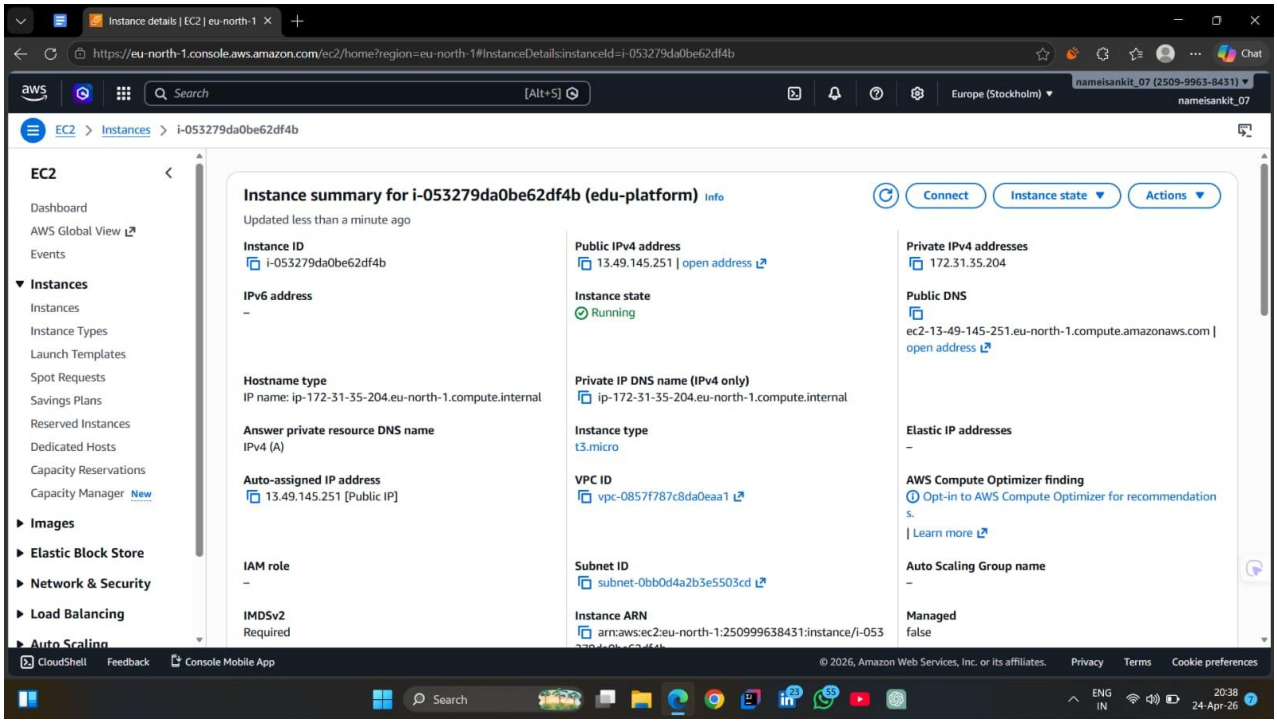

github.com/nameisankit/EduPlatform-AI/blob/main/.github/workflows/cicd.yml
main
EduPlatform-AI / .github / workflows / cicd.yml
Top
Code
Blame
Raw
- name: Push image
run: docker push ${{ secrets.DOCKER_USERNAME }}/edu-platform:latest
- name: Deploy to EC2
uses: appleboy/ssh-action@master
with:
host: ${{ secrets.EC2_HOST }}
username: ubuntu
key: ${{ secrets.EC2_KEY }}
script: |
docker system prune -a -f
docker stop eduplatform || true
docker rm eduplatform || true
docker pull ${{ secrets.DOCKER_USERNAME }}/edu-platform:latest
docker run -d -p 8501:8501 \
--name eduplatform \
--restart always \
-e GROQ_API_KEY=${{ secrets.GROQ_API_KEY }} \
${{ secrets.DOCKER_USERNAME }}/edu-platform:latest

5.3 AWS EC2 Deployment

The production deployment of EduPlatform AI is hosted on an AWS EC2 t2.micro instance running Ubuntu 22.04 LTS. The instance is located in the ap-south-1 (Mumbai) region to minimize latency for users in India. The following configuration steps were performed to prepare the EC2 instance:

- Launch EC2 instance with Ubuntu 22.04 AMI and t2.micro instance type.
- Configure Security Group to allow inbound traffic on port 22 (SSH), port 80 (HTTP), and port 8501 (Streamlit).
- Install Docker and Docker Compose on the instance.
- Configure the instance to start Docker on boot using systemd.
- Add the deploy.sh deployment script to the instance and configure GitHub Actions SSH access.
- Set up a static Elastic IP address to ensure the application URL remains constant across instance restarts.

The application is accessible via the EC2 instance's public IP address on port 8501. Future enhancements include configuring a custom domain name and SSL certificate using AWS Route 53 and Let's Encrypt, enabling HTTPS access.

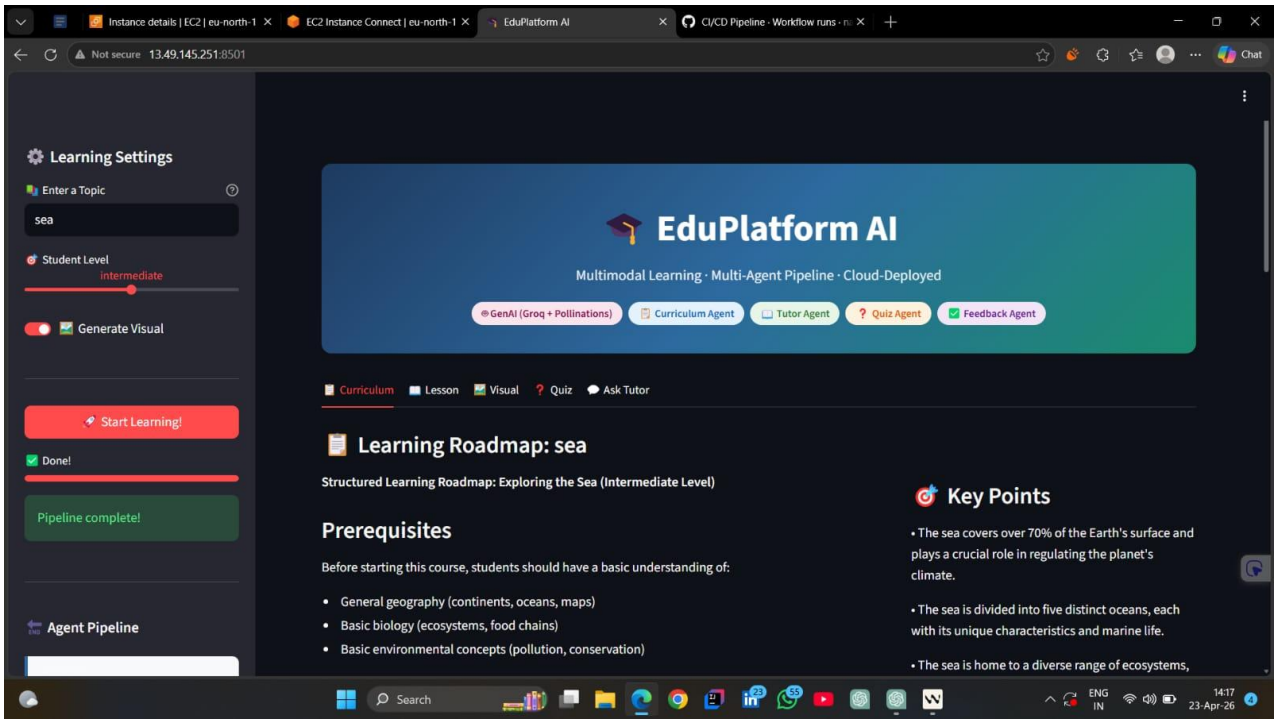


Chapter 6

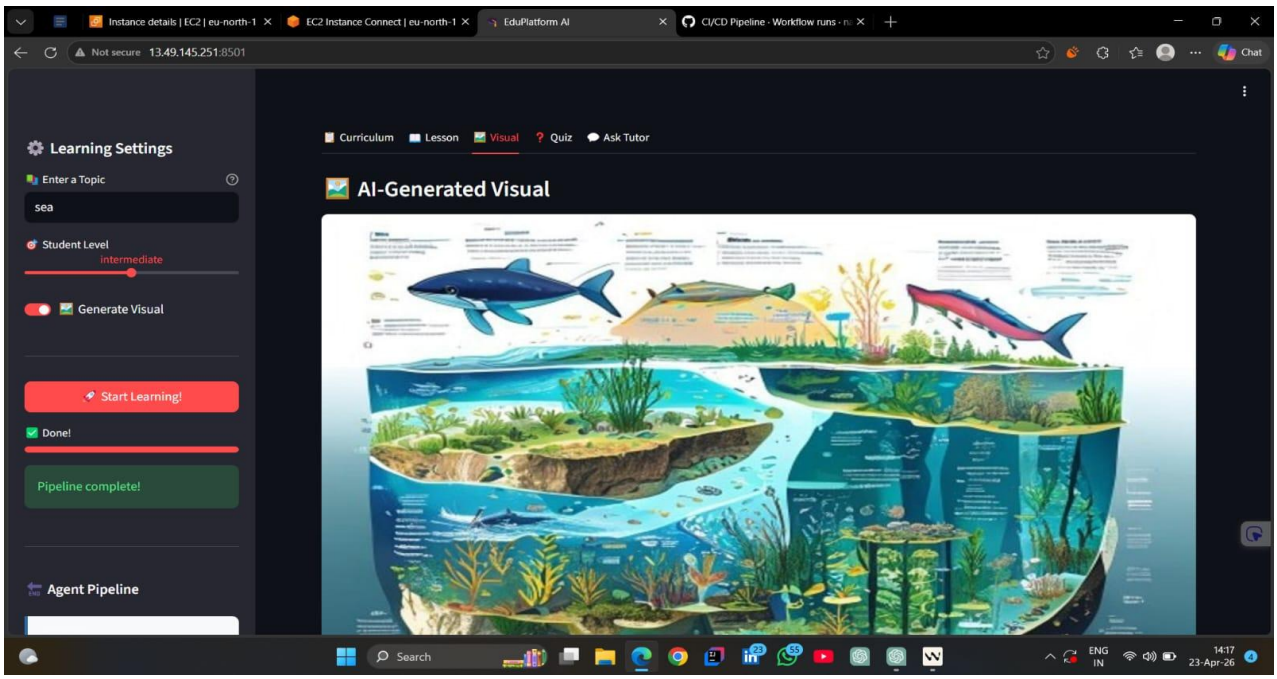
Results and Discussions

6.1 System Output

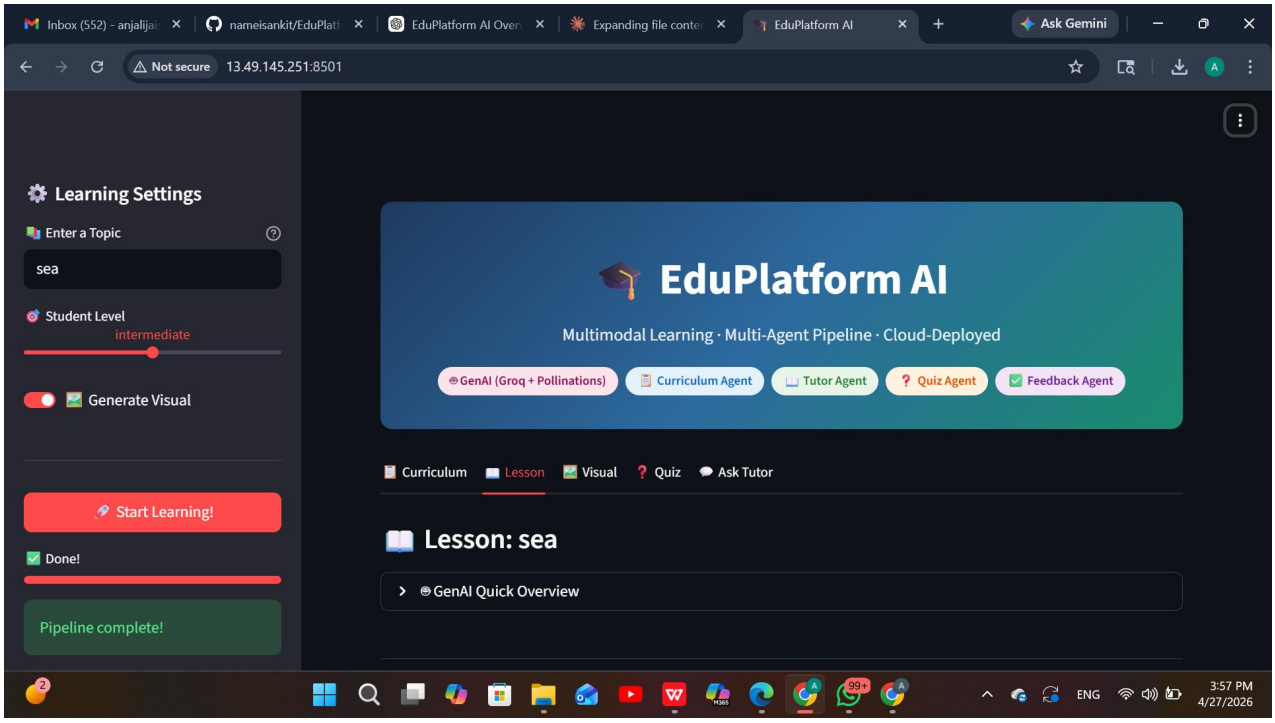
Curriculum

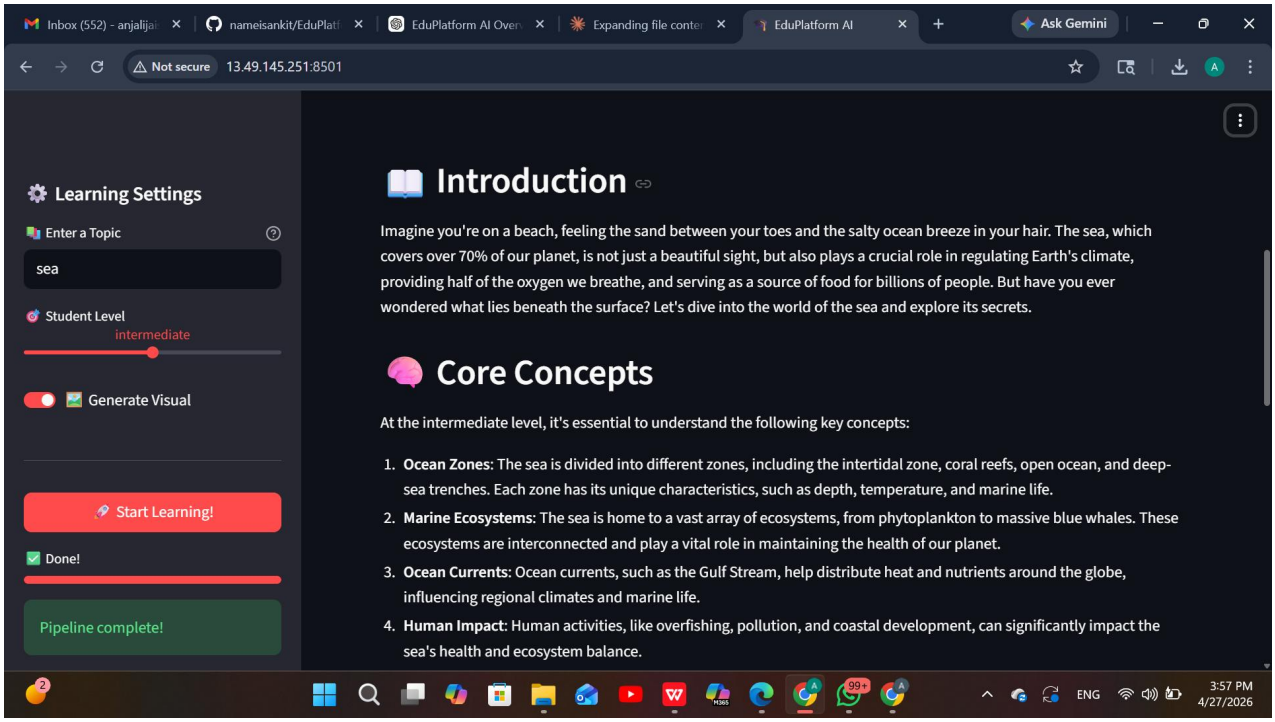


Visual

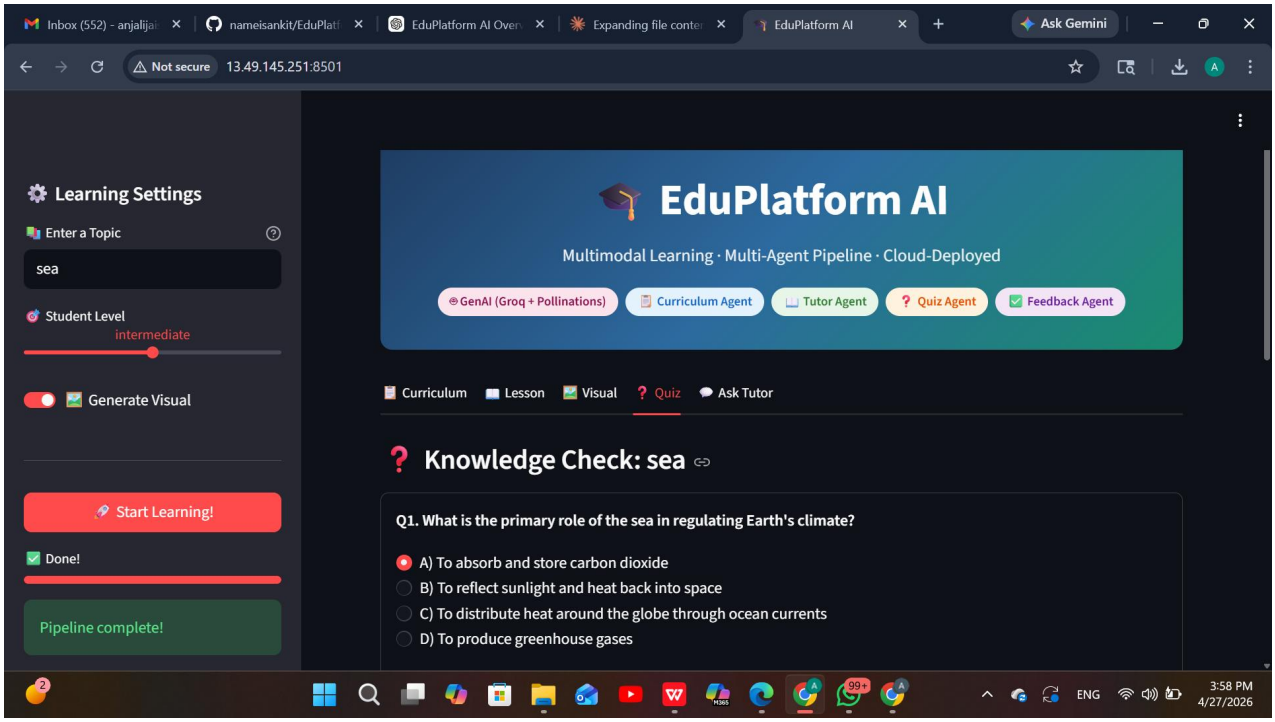


Lesson

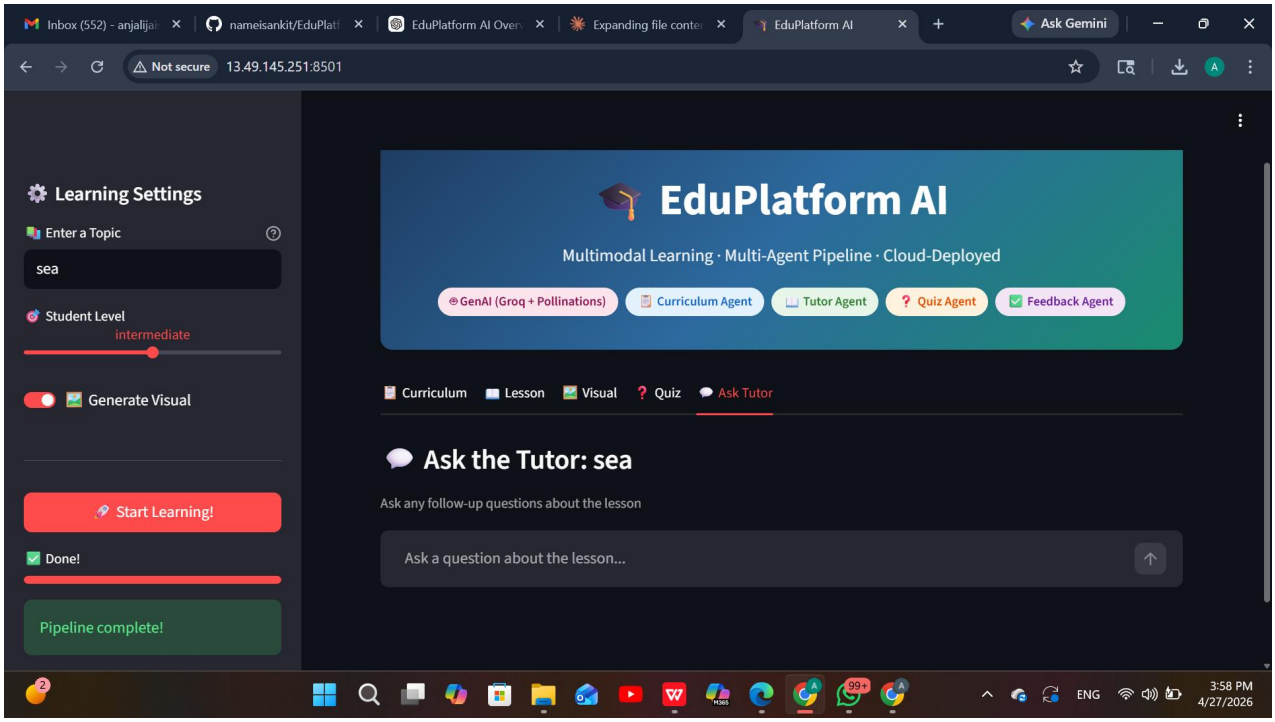

Inbox (552) - anjalija
nameisankit/EduPlatf
EduPlatform AI Over
Expanding file conte
EduPlatform AI
Ask Gemini
Not secure
13.49.145.251:8501
Learning Settings
Enter a Topic
sea
Student Level
intermediate
Generate Visual
Start Learning!
Done!
Pipeline complete!
EduPlatform AI
Multimodal Learning · Multi-Agent Pipeline · Cloud-Deployed
GenAI (Groq + Pollinations)
Curriculum Agent
Tutor Agent
Quiz Agent
Feedback Agent
Curriculum
Lesson
Visual
Quiz
Ask Tutor
Lesson: sea
GenAI Quick Overview
ENG
3:57 PM
4/27/2026


Inbox (552) - anjalija
nameisankit/EduPlatf
EduPlatform AI Over
Expanding file conte
EduPlatform AI
Ask Gemini
Not secure
13.49.145.251:8501
Learning Settings
Enter a Topic
sea
Student Level
intermediate
Generate Visual
Start Learning!
Done!
Pipeline complete!
Introduction
Imagine you're on a beach, feeling the sand between your toes and the salty ocean breeze in your hair. The sea, which covers over 70% of our planet, is not just a beautiful sight, but also plays a crucial role in regulating Earth's climate, providing half of the oxygen we breathe, and serving as a source of food for billions of people. But have you ever wondered what lies beneath the surface? Let's dive into the world of the sea and explore its secrets.
Core Concepts
At the intermediate level, it's essential to understand the following key concepts:
1. Ocean Zones: The sea is divided into different zones, including the intertidal zone, coral reefs, open ocean, and deep-sea trenches. Each zone has its unique characteristics, such as depth, temperature, and marine life.
2. Marine Ecosystems: The sea is home to a vast array of ecosystems, from phytoplankton to massive blue whales. These ecosystems are interconnected and play a vital role in maintaining the health of our planet.
3. Ocean Currents: Ocean currents, such as the Gulf Stream, help distribute heat and nutrients around the globe, influencing regional climates and marine life.
4. Human Impact: Human activities, like overfishing, pollution, and coastal development, can significantly impact the sea's health and ecosystem balance.
ENG
3:57 PM
4/27/2026

Quiz



Ask Tutor



6.2 Challenges Faced

During the development and deployment of the EduPlatform AI system, several challenges were encountered. One of the major challenges was integrating the Groq API with the multi-agent pipeline, ensuring that the generated outputs were consistent and properly passed between agents. Managing the sequential flow of data between curriculum, tutor, quiz, and feedback agents required careful orchestration.The development and deployment of EduPlatform AI presented several technical and design challenges:

6.3.1 LLM Prompt Engineering

Crafting effective prompts for each agent required extensive experimentation. Prompts that were too general produced loosely structured outputs that were difficult to parse programmatically. Prompts that were too prescriptive sometimes caused the LLM to deviate from the specified format. A hybrid approach — combining structured instructions with output format templates — was found to produce the most consistently parseable and pedagogically sound outputs.

6.3.2 Context Window Management

For complex topics with large curricula, the total context (curriculum + explanations) could approach or exceed the LLM's context window limit (8192 tokens for llama3-70b-8192). A sliding window strategy was implemented for the Quiz and Feedback agents, prioritizing the most relevant subtopics when constructing prompts for topics with large curricula.

6.3.3 API Reliability

The Groq API, while generally reliable, experienced occasional rate-limiting and transient failures during development. Exponential backoff retry logic and user-facing loading indicators mitigated the impact of these failures on the user experience. A fallback mechanism was implemented to use cached responses for repeated queries.

6.3.4 Docker Build Optimization

Initial Docker builds were slow (~8 minutes) due to large dependency installation times. Docker layer caching was optimized by restructuring the Dockerfile to copy requirements.txt before copying the application code, enabling the dependency layer to be cached across builds that modify only the application code. This reduced incremental build times to approximately 30 seconds.

Chapter 7

Summary and Conclusions

7.1 Summary

The EduPlatform AI project presents a comprehensive and intelligent learning system that integrates Generative AI, multi-agent architecture, and cloud-based deployment to deliver an automated educational experience. The platform allows users to input a topic along with a difficulty level, based on which it dynamically generates a structured learning roadmap, detailed explanations, visual content, quiz questions, and feedback. The system is built using Python and leverages **Streamlit** for creating an interactive user interface. The core intelligence of the system is powered by the **Groq**, which enables high-speed and efficient generation of textual educational content, along with image generation using Pollinations AI.

A key highlight of the project is the implementation of a multi-agent pipeline, where different agents such as curriculum, tutor, quiz, and feedback agents work collaboratively to process and refine the content in a sequential manner. The system also incorporates modern DevOps practices, including containerization using Docker and automated deployment using **GitHub Actions**. The application is deployed on an Amazon Web Services EC2 instance, making it accessible through a web interface. This end-to-end integration of AI, software engineering, and cloud technologies makes the platform scalable, efficient, and suitable for real-world educational applications.

In addition to these features, the project successfully demonstrates the practical application of advanced AI techniques in solving real-world challenges in the education domain. By automating the generation of educational content, the system significantly reduces the dependency on manual content creation while ensuring consistency and adaptability across different topics and difficulty levels. The integration of multimodal learning further enhances user engagement by combining textual explanations with relevant visual representations, thereby improving comprehension and retention.

The modular design of the multi-agent system ensures that each component operates independently while contributing to the overall workflow, making the system highly maintainable and extensible. This architecture allows for easy integration of future enhancements such as personalized learning paths, adaptive difficulty adjustment, and advanced analytics for tracking learner progress.The project also highlights the importance of combining AI with modern software engineering practices. By utilizing containerization and cloud deployment, the system achieves portability, scalability, and ease of maintenance, making it suitable for production environments. This demonstrates how AI-driven applications can be effectively deployed and managed using DevOps methodologies.

Overall, EduPlatform AI serves as a robust and scalable intelligent tutoring system that showcases the seamless integration of Generative AI, multi-agent systems, and cloud technologies. It provides a strong foundation for future research and development in AI-based education systems and represents a significant step toward creating adaptive, automated, and user-centric learning platforms.

7.2 Conclusion

The EduPlatform AI system successfully demonstrates the integration of Generative AI, multi-agent architecture, and modern DevOps practices to create an intelligent and automated learning platform. By leveraging **Streamlit** for the user interface and the **Groq** API for content generation, the system is able to dynamically generate structured educational material, including lessons, visuals, quizzes, and feedback. The use of a multi-agent pipeline ensures modularity and efficient task execution, making the system scalable and easy to extend.

Furthermore, the implementation of containerization using Docker and deployment on Amazon Web Services EC2, along with CI/CD automation through **GitHub Actions**, highlights the practical application of DevOps in real-world projects. The system achieves reliable performance, automated deployment, and ease of maintenance. Overall, the project provides a strong foundation for building advanced AI-driven educational platforms and can be further enhanced with features such as user authentication, progress tracking, and scalability improvements.

The project makes several original contributions to the field: (i) a novel multi-agent pipeline architecture for educational content generation, where each agent's output contextually enriches the inputs of subsequent agents; (ii) a practical integration pattern for combining LLM-based text generation with diffusion-based image generation in a unified educational context; and (iii) a reference implementation demonstrating how GenAI-powered applications can be containerized and deployed using industry-standard DevOps practices.

The challenges encountered — including LLM context window management, prompt engineering complexity, and API reliability — have yielded practical solutions that can inform future work in AI application development. The comparative analysis confirms that EduPlatform AI occupies a unique position in the EdTech landscape, combining dynamic content generation, structured learning paths, multimodal delivery, and automated assessment in a single, integrated platform that no existing solution currently offers.

In conclusion, EduPlatform AI represents a meaningful step toward the vision of democratized, personalized education powered by AI — a future in which every learner has access to a knowledgeable, adaptive tutor at any time, on any topic, at any difficulty level.

Chapter 8

Future scope

Future scope

- Implement user authentication and profile management for personalized learning
- Add database integration to store user progress, quiz results, and history
- Introduce adaptive learning to customize content based on user performance
- Optimize API performance of **Groq** integration
- Strengthen security and data handling mechanisms
- Improve real-time feedback and evaluation system
- Enhance UI/UX of the **Streamlit** interface

Appendix

Appendix A: Project Source Code Structure

The complete project is organized as follows:

```
edu-platform/
├── app.py                      # 🎯 Main Streamlit app (entry point)
├── requirements.txt            # 📦 Python dependencies
├── Dockerfile                  # 🐳 Docker build config
├── docker-compose.yml          # ⚙️ Multi-container setup
├── run.bat                     # 🪟 Windows one-click run
├── run.sh                      # 🐧 Mac/Linux run script
├── .env                        # 🔐 Environment variables (not pushed)
├── .gitignore
│
├── core/                       # 🧠 GenAI Layer
│   ├── llm.py                  # Groq LLM integration (text generation)
│   └── image_gen.py            # Pollinations image generation (no API key)
│
├── agents/                     # 🤖 Multi-Agent System
│   ├── base.py                 # Base agent class
│   ├── education_agents.py     # Curriculum, Tutor, Quiz, Feedback agents
│   └── pipeline.py             # Agent orchestration pipeline
│
├── .github/
│   └── workflows/
│       └── cicd.yml            # 🚀 CI/CD pipeline (build + deploy)
│
├── venv/                       # ⚠️ Local virtual environment
```

Appendix B: Sample Environment Variables

The system requires API keys and configuration stored in .env file:

GROQ_API_KEY=your_groq_api_key_here

Appendix C: Multi-Agent Workflow

🧠 Agent Pipeline Flow

```
User Input (Topic + Level)
        ↓
📋 Curriculum Agent → Generates roadmap
        ↓
📖 Tutor Agent → Generates explanation
        ↓
🖼️ Image Generator → Creates visuals
        ↓
❓ Quiz Agent → Generates questions
        ↓
✅ Feedback Agent → Evaluates answers
        ↓
📊 Final Output (Streamlit UI)
```

Appendix D: Key Modules Description

1. llm.py

- Connects to Groq LLM API
- Generates educational content

2. image_gen.py

- Uses Pollinations AI
- Generates topic-based images without API key

3. education_agents.py

- Contains all AI agents
- Implements reasoning + role-based behavior

4. pipeline.py

- Orchestrates all agents
- Ensures sequential execution flow

Appendix E: Deployment Details

Docker Deployment

- Application containerized using Docker
- Exposed on port 8501

AWS Deployment

- EC2 instance used
- Docker container runs in background
- Public IP used for access

Bibliography

1. OpenAI Documentation — https://platform.openai.com/docs
   (Reference for Large Language Model concepts and prompt engineering)
2. Groq AI Documentation — https://console.groq.com/docs
   (Used for fast LLM inference and text generation integration)
3. Streamlit Documentation — https://docs.streamlit.io
   (Used for building the interactive web application UI)
4. Docker Documentation — https://docs.docker.com
   (Used for containerization and deployment of the application)
5. AWS EC2 Documentation — https://docs.aws.amazon.com/ec2
   (Used for cloud deployment and hosting of the project)
6. GitHub Actions Documentation — https://docs.github.com/actions
   (Used for implementing CI/CD pipeline automation)
7. Pollinations AI — https://pollinations.ai
   (Used for AI-based image generation without API key.

GitHub Link : https://github.com/nameisankit/EduPlatform-AI

Live Demo : http://13.49.145.251:8501